

開始使用 gRPC-Java

來源：<https://codelabs.developers.google.com/grpc/getting-started-grpc-java?hl=zh-tw>

步驟數：8

瞭解如何搭配使用通訊協定緩衝區編譯器和 gRPC-Java，以 Java 程式設計語言建構 gRPC 用戶端伺服器應用程式。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 定義服務](#)
4. [4. 產生用戶端和伺服器程式碼](#)
5. [5. 導入伺服器](#)
6. [6. 建立用戶端](#)
7. [7. 快來體驗看看吧！](#)
8. [8. 後續步驟](#)

1. 簡介

在本程式碼研究室中，您將使用 gRPC-Java 建立用戶端和伺服器，做為以 Java 編寫的路線對應應用程式基礎。

完成本教學課程後，您將擁有一個用戶端，可使用 [gRPC](#) 連線至遠端伺服器，取得地圖上特定座標位置的名稱或郵寄地址。完整的應用程式可能會使用這種用戶端/伺服器設計，列舉或摘要說明路線上的興趣點。

伺服器的 API 定義於通訊協定緩衝區檔案中，可用於產生用戶端和伺服器的樣板程式碼，以便彼此通訊，節省您實作該功能的時間和精力。

產生的程式碼不僅會處理伺服器與用戶端之間複雜的通訊，也會處理資料序列化和還原序列化。

課程內容

- 如何使用通訊協定緩衝區定義服務 API。
- 如何使用自動程式碼生成功能，從通訊協定緩衝區定義建構以 gRPC 為基礎的用戶端和伺服器。
- 瞭解如何透過 gRPC 進行用戶端與伺服器之間的通訊。

本程式碼研究室適用於剛接觸 gRPC 或想複習 gRPC 的 Java 開發人員，以及有興趣建構分散式系統的任何人。不需要有 gRPC 相關經驗。

步驟 2 · 約 0 分鐘

2. 事前準備

必要條件

- [JDK](#) 第 8 版或更高版本

取得程式碼

為避免您必須從頭開始，本程式碼研究室提供應用程式的原始碼架構，供您完成。下列步驟將說明如何完成應用程式，包括使用通訊協定緩衝區編譯器外掛程式產生樣板 gRPC 程式碼。

首先，請建立程式碼研究室工作目錄，然後 cd 到該目錄：

```
mkdir grpc-java-getting-started && cd grpc-java-getting-started
```

下載並擷取程式碼研究室：



```
curl -sL https://github.com/grpc-ecosystem/grpc-codelabs/archive/refs/heads/v1.tar.gz \  
| tar xvz --strip-components=4 \  
grpc-codelabs-1/codelabs/grpc-java-getting-started/start_here
```

或者，您也可以下載只包含 Codelab 目錄的 .zip 檔案，然後手動解壓縮。

如要略過實作的輸入作業，可以在 [GitHub](#) 上取得完整的原始碼。

3. 定義服務

首先，您要使用[通訊協定緩衝區](#)，定義應用程式的 gRPC 服務、RPC 方法，以及要求和回應訊息類型。你的服務將提供：

- 伺服器實作且用戶端呼叫的 RPC 方法 `GetFeature`。
- 訊息類型 `Point` 和 `Feature` 是使用 `GetFeature` 方法時，用戶端與伺服器之間交換的資料結構。用戶端會在傳送至伺服器的 `GetFeature` 要求中，以 `Point` 形式提供地圖座標，而伺服器會回覆相應的 `Feature`，說明這些座標所指位置的內容。

這個 RPC 方法及其訊息類型都會在提供的原始碼 `src/main/proto/routeguide/route_guide.proto` 檔案中定義。

通訊協定緩衝區通常稱為 `protobuf`。如要進一步瞭解 gRPC 術語，請參閱 gRPC 的「[核心概念、架構和生命週期](#)」。

由於我們要在這個範例中產生 Java 程式碼，因此已在 `.proto` 中指定 `java_package` 檔案選項和 Java 類別的名稱：

```
option java_package = "io.grpc.examples.routeguide";
option java_outer_classname = "RouteGuideProto";
```

訊息類型

在原始碼的 `routeguide/route_guide.proto` 檔案中，請先定義 `Point` 訊息型別。`Point` 代表地圖上的經緯度座標組合。在本程式碼研究室中，請使用整數做為座標：

```
message Point {
  int32 latitude = 1;
  int32 longitude = 2;
}
```

數字 1 和 2 是 `message` 結構中每個欄位的專屬 ID 號碼。

接著定義 `Feature` 訊息類型。`Feature` 使用 `string` 欄位，為 `Point` 指定位置的某個項目提供名稱或郵寄地址：

```
message Feature {
  // The name or address of the feature.
  string name = 1;

  // The point where the feature is located.
  Point location = 2;
}
```

服務方法

`route_guide.proto` 檔案具有名為 `RouteGuide` 的 `service` 結構，可定義應用程式服務提供的一或多個方法。

在 `RouteGuide` 定義中新增 `rpc` 方法 `GetFeature`。如先前所述，這個方法會根據一組指定座標查詢地點名稱或地址，因此請讓 `GetFeature` 為指定 `Point` 傳回 `Feature`：

```
service RouteGuide {  
    // Definition of the service goes here  
  
    // Obtains the feature at a given position.  
    rpc GetFeature(Point) returns (Feature) {}  
}
```

這是 unary RPC 方法：簡單的 *RPC*，用戶端會將要求傳送至伺服器，並等待伺服器傳回回應，就像呼叫本機函式一樣。

4. 產生用戶端和伺服器程式碼

接下來，我們需要從 `.proto` 服務定義產生 gRPC 用戶端和伺服器介面。我們會使用通訊協定緩衝區編譯器 `protoc` 和特殊的 gRPC Java 外掛程式來完成這項作業。您必須使用 `proto3` 編譯器 (同時支援 `proto2` 和 `proto3` 語法)，才能產生 gRPC 服務。

使用 Gradle 或 Maven 時，`protoc` 建構外掛程式可以在建構過程中產生必要程式碼。如要瞭解如何從自己的 `.proto` 檔案產生程式碼，請參閱 [grpc-java README](#)。

我們已在程式碼研究室的原始碼中提供 Gradle 環境和設定，用來建構這個專案。

在 `grpc-java-getting-started` 目錄中執行下列指令：

```
$ chmod +x gradlew
$ ./gradlew generateProto
```

下列類別是從服務定義產生：

- `Feature.java`、`Point.java` 和其他項目，其中包含所有通訊協定緩衝區程式碼，可填入、序列化及擷取要求和回應訊息類型。
 - `RouteGuideGrpc.java`，其中包含 (以及一些其他實用程式碼) 供 `RouteGuide` 伺服器實作的基本類別 `RouteGuideGrpc.RouteGuideImplBase`，以及在 `RouteGuide` 服務中定義的所有方法，和供用戶端使用的存根類別。
-

5. 導入伺服器

首先，我們來看看如何建立 `RouteGuide` 伺服器。如要讓 `RouteGuide` 服務正常運作，需要完成以下兩個步驟：

- 實作從服務定義產生的服務介面，執行服務的實際「工作」。
- 執行 gRPC 伺服器，監聽來自用戶端的要求，並將要求分派至正確的服務實作。

實作 `RouteGuide`

如您所見，我們的伺服器有一個 `RouteGuideService` 類別，可擴充產生的 `RouteGuideGrpc.RouteGuideImplBase` 抽象類別：

```
private static class RouteGuideService extends RouteGuideGrpc.RouteGuideImplBase {  
    ...  
}
```

我們提供下列 2 個檔案，用於初始化伺服器功能：

```
./src/main/java/io/grpc/examples/routeguide/RouteGuideUtil.java
```

```
./src/main/resources/io/grpc/examples/routeguide/route_guide_db.json
```

現在來詳細瞭解簡單的 RPC 實作方式。

Unary RPC

`RouteGuideService` 會實作所有服務方法。在本例中，這只是 `GetFeature()`，會從用戶端接收 `Point` 訊息，並從已知地點清單中傳回對應的位置資訊 (以 `Feature` 訊息的形式)。

```
@Override  
public void getFeature(Point request, StreamObserver<Feature> responseObserver) {  
    responseObserver.onNext(checkFeature(request));  
    responseObserver.onCompleted();  
}
```

`getFeature()` 方法會採用兩個參數：

- `Point`：要求。
- `StreamObserver<Feature>`：回應觀察器，這是伺服器用來呼叫回應的特殊介面。

如要將回應傳回給用戶端並完成呼叫，請執行下列操作：

1. 我們會建構並填入 `Feature` 回應物件，然後傳回給用戶端，如服務定義中所述。在本範例中，我們會在個別的私有 `checkFeature()` 方法中執行這項操作。
2. 我們使用回應觀察器的 `onNext()` 方法傳回 `Feature`。
3. 我們使用回應觀察器的 `onCompleted()` 方法，指定我們已完成處理 RPC。

啟動伺服器

實作所有服務方法後，我們需要啟動 gRPC 伺服器，用戶端才能實際使用服務。我們在樣板中加入 `ServerBuilder` 物件的建立作業：


```
ServerBuilder.forPort(port), port, RouteGuideUtil.parseFeatures(featureFile)
```

我們在建構函式中建構服務：

1. 使用建構工具的 `forPort()` 方法，指定要用來監聽用戶端要求的通訊埠 (系統會使用萬用字元位址)。
2. 建立服務實作類別 `RouteGuideService` 的例項，並傳遞至建構工具的 `addService()` 方法。
3. 在建構工具上呼叫 `build()`，為我們的服務建立 RPC 伺服器。

以下程式碼片段顯示如何建立 `ServerBuilder` 物件。

```
/** Create a RouteGuide server listening on {@code port} using {@code featureFile} database. */
public RouteGuideServer(int port, URL featureFile) throws IOException {
    this(Grpc.newServerBuilderForPort(port, InsecureServerCredentials.create()),
        port, RouteGuideUtil.parseFeatures(featureFile));
}
```

下列程式碼片段顯示如何為 `RouteGuide` 服務建立伺服器物件。

```
/** Create a RouteGuide server using serverBuilder as a base and features as data. */
public RouteGuideServer(ServerBuilder<?> serverBuilder, int port, Collection<Feature>
features) {
    this.port = port;
    server = serverBuilder.addService(new RouteGuideService(features))
        .build();
}
```

實作啟動方法，在上述建立的伺服器上呼叫 `start`。

```
public void start() throws IOException {
    server.start();
    logger.info("Server started, listening on " + port);
}
```

實作等待伺服器完成作業的方法，避免立即結束。

```
/** Await termination on the main thread since the grpc library uses daemon threads. */
private void blockUntilShutdown() throws InterruptedException {
    if (server != null) {
        server.awaitTermination();
    }
}
```

如您所見，我們使用 `ServerBuilder` 建構及啟動伺服器。

在主要方法中，我們會：

1. 建立 `RouteGuideServer` 執行個體。
2. 呼叫 `start()`，為我們的服務啟用 RPC 伺服器。
3. 呼叫 `blockUntilShutdown()`，等待服務停止。

```
public static void main(String[] args) throws Exception {  
    RouteGuideServer server = new RouteGuideServer(8980);  
    server.start();  
    server.blockUntilShutdown();  
}
```

步驟 6 · 約 5 分鐘

6. 建立用戶端

在本節中，我們將瞭解如何為 `RouteGuide` 服務建立用戶端。

建立虛設常式

若要呼叫服務方法，我們必須先建立 *stub*。有兩種型別的 `Stub`，但本程式碼研究室只需要使用封鎖型別。這兩種類型分別是：

- *封鎖/同步存根*，會發出 `RPC` 呼叫並等待伺服器回應，然後傳回回應或引發例外狀況。
- *非阻塞/非同步存根*，可對伺服器發出非阻塞呼叫，並以非同步方式傳回回應。您只能使用非同步存根撥打特定類型的串流呼叫。

首先，我們需要建立 `gRPC` 管道，然後使用該管道建立存根。

我們可以直接使用 `ManagedChannelBuilder` 建立管道。

```
ManagedChannelBuilder.forAddress(host, port).usePlaintext().build
```

但我們來使用會採用含 `hostname:port` 的字串的公用程式方法。

```
Grpc.newChannelBuilder(target, InsecureChannelCredentials.create()).build();
```

現在，我們可以透過管道建立封鎖存根。在本程式碼研究室中，我們只有封鎖 `RPC`，因此使用從 `.proto` 產生的 `RouteGuideGrpc` 類別中提供的 `newBlockingStub` 方法。

```
blockingStub = RouteGuideGrpc.newBlockingStub(channel);
```

呼叫服務方法

接著來看看如何呼叫服務方法。

簡單 `RPC`

呼叫簡單的 `RPC` `GetFeature` 幾乎與呼叫本機方法一樣簡單。

我們會建立並填入要求通訊協定緩衝區物件 (在本範例中為 `Point`)，將其傳遞至封鎖存根上的 `getFeature()` 方法，然後取得 `Feature`。

如果發生錯誤，系統會將錯誤編碼為 `Status`，我們可以從 `StatusRuntimeException` 取得該錯誤。

```
Point request = Point.newBuilder().setLatitude(lat).setLongitude(lon).build();

Feature feature;
try {
    feature = blockingStub.getFeature(request);
} catch (StatusRuntimeException e) {
    logger.log(Level.WARNING, "RPC failed: {0}", e.getStatus());
    return;
}
```

樣板會記錄一則訊息，其中包含的內容取決於指定點是否有特徵。

步驟 7 · 約 2 分鐘

7. 快來體驗看看吧！

1. 在 `start_here` 目錄中執行下列指令：

```
$ ./gradlew installDist
```

這會編譯您的程式碼、將其封裝在 JAR 中，並建立執行範例的指令碼。這些檔案會建立在 `build/install/start_here/bin/` 目錄中。腳本為：`route-guide-server` 和 `route-guide-client`。

啟動用戶端前，伺服器必須先執行。

2. 執行伺服器：

```
$ ./build/install/start_here/bin/route-guide-server
```

3. 執行用戶端：

```
$ ./build/install/start_here/bin/route-guide-client
```

您會看到類似下列的輸出內容，為求清楚起見，時間戳記已省略：

```
INFO: *** GetFeature: lat=409,146,138 lon=-746,188,906
INFO: Found feature called "Berkshire Valley Management Area Trail, Jefferson, NJ,
USA" at 40.915, -74.619
INFO: *** GetFeature: lat=0 lon=0
INFO: Found no feature at 0, 0
```

步驟 8 · 約 0 分鐘

8. 後續步驟

- 請參閱「[gRPC 簡介](#)」和「[核心概念](#)」，瞭解 gRPC 的運作方式
 - 逐步完成[基礎教學課程](#)
 - 探索 [API 參考資料](#)。
-